
Hessian approximations beyond optimization

Anonymous Author(s)

Affiliation

Address

email

Abstract

Hessian approximations are a foundational tool in neural network optimization, where they enable efficient second-order methods. However, an increasing number of applications beyond optimization, such as influence functions, unrolling, and loss landscape analysis, rely on the same approximations under fundamentally different requirements. In optimization, approximations are used and recomputed frequently, making computational cost the essential constraint. On the other hand, in emerging applications, the curvature approximation is computed once and used many times, shifting the focus from speed to accuracy. So far, current methods have been inherited from the optimization setting and have not been systematically evaluated for accuracy. In this paper, we examine what makes a Hessian approximation “accurate” by developing a suite of metrics, including downstream performance measures, that go beyond standard matrix approximation quality. We introduce and benchmark a wider range of approximations than are commonly used, such as (E)K-FAC and Shampoo. We find that performance across our metrics is largely decoupled: matrix approximation quality does not predict downstream performance, and results are highly sensitive to hyperparameter choices such as the damping coefficient. Our results suggest that better downstream performance may arise for reasons other than being a faithful approximation of the Hessian.

1 Introduction

Hessian approximations are widely used in optimization, where they are often used to compute the natural gradient [Amari, 1998]. While it has been observed that they enable better optimization performance than first order methods, their use has been limited by their computational cost. As such, computational efficiency has been a key consideration when developing and evaluating Hessian approximations, like (E)K-FAC [Martens and Grosse, 2015, George et al., 2018] and Shampoo [Gupta et al., 2018].

More recently, Hessian approximations have enjoyed increased attention as a tool to understand the geometry of the loss landscape and its connection to the training data, especially at the scale of large language models [Grosse et al., 2023]. The three main applications we will focus on in this work are:

1. Influence functions, which require inverse Hessian-vector products (iHVPs) to approximate the effect of removing a training example on a model’s weights and its downstream behavior.
2. Approximate unrolling, where we require Hessian-vector products (HVPs) to compute the Jacobian of the final network parameters w.r.t. the parameters at an earlier timestep [Bae et al., 2024].
3. Weight editing to ameliorate memorization [Merullo et al., 2025].

So far, most literature has used (E)K-FAC as the default Hessian approximation for these applications [Bae et al., 2022, Grosse et al., 2023, Ruis et al., 2025]. However, it is not clear that this is the best

choice, especially at the scale of large language models where the computational cost of EK-FAC explodes. This raises two questions: 1. Are there better alternatives? 2. How exactly do we measure the quality of a Hessian approximation? We discuss a range of metrics that have been used as heuristics, such as the Frobenius distance between the approximate and true Hessians [George et al., 2018], the average error incurred by the approximate Hessian to compute iHVPs, or the downstream performance on a task that uses the Hessian approximation. We find that these metrics often correlate very poorly with one another, and that approximations can perform well on downstream tasks despite performing poorly on other metrics. We hypothesize an explanation for this phenomenon and discuss its implications. We suggest possible new approaches to Hessian approximations that may be more suitable for applications outside optimization.

Our key contributions are as follows:

1. We introduce a benchmark of Hessian approximations at three scales, from small models where the exact Hessian is computable, to large language models. We evaluate a range of methods and study their sensitivity to hyperparameters such as sample size and the damping coefficient.
2. We find that downstream performance often poorly correlates with the mathematical accuracy of a Hessian approximation, and we provide a possible hypothesis for this phenomenon.
3. We observe that simpler methods outperform more complex methods such as EK-FAC, and we discuss the implications of this finding for the design of Hessian approximations.
4. We release our code and benchmark to facilitate future research on Hessian approximations beyond optimization.

2 Related work

Hessian approximations in optimization. Hessian approximations originated in second-order optimization, where the Fisher information matrix serves as a tractable curvature proxy [Amari, 1998]. To scale to modern networks, Kronecker-factored approximations have been proposed: K-FAC [Martens and Grosse, 2015] factorizes each layer’s Fisher block into activation and pseudo-gradient covariances, EK-FAC [George et al., 2018] adds an eigenvalue correction, and Shampoo [Gupta et al., 2018] – equivalent to TK-FAC [Gao et al., 2021, Morwani et al., 2025] – gives the optimal rank-one Kronecker approximation in Frobenius norm. Designed as preconditioners, these methods are judged by convergence speed rather than curvature fidelity, and recent work questions whether their gains stem from faithful curvature modeling at all: Benzing [2022] show K-FAC’s performance is driven by damping that collapses the pseudo-gradient factor to the identity, yielding FOOF, which matches K-FAC at lower cost; Seung et al. [2025] simplify this further to a rank-one outer product of mean activations.

Hessian approximations beyond optimization. Hessian approximations have been adopted in settings where curvature is built once and queried many times. Influence functions [Koh and Liang, 2017, Grosse et al., 2023] require inverse-Hessian-vector products to estimate training-example impact. Unrolling methods like SOURCE [Bae et al., 2024] use Hessian-vector products to amortize differentiation through the training trajectory, and Merullo et al. [2025] use K-FAC eigendecomposition to suppress directions associated with verbatim memorization. Across these settings, (E)K-FAC has become the de facto default, despite limited systematic comparison against alternative approximations.

Evaluating Hessian approximations. Work examining approximation error in these applications remains limited. Kunstner et al. [2019] draw an important distinction between the empirical and true Fisher, which is directly relevant given that the GNH-Fisher equivalence we exploit in Appendix E holds only in expectation over model-sampled labels. Bae et al. [2022] show that influence functions do not faithfully recover their underlying leave-one-out retraining objective, decomposing the gap into five irreducible error components. Most empirical comparisons in the influence-function literature consider only LiSSA and EK-FAC [Grosse et al., 2023, Zhu and Cangelosi, 2025]. Wang et al. [2026b] conduct a study of hyperparameter sensitivity across multiple data attribution methods, showing that influence-function methods are highly sensitive to the choice of regularization parameter on the inverse Hessian. Closest to our work, Wang et al. [2026a] relate curvature approximation error

to attribution quality, but only at a single level of application and limited scale. In contrast, in this work we evaluate approximation quality jointly across applications, metrics, and scales.

3 Hessians in the wild

We set the notation for the rest of the paper: We let $\mathcal{D} = \{x_i, y_i\}_{i=1}^n$ be the training dataset and $\mathbf{w} \in W \subset \mathbb{R}^p$ be the parameters of a neural network which induce a function $f_{\mathbf{w}} : X \rightarrow Y$. We denote by $\beta = \{\beta_i\}_{i=1}^n \subset [0, 1]^n$ the weights of the data points, meaning that we consider generalized loss functions

$$\mathcal{L}(\mathbf{w}, \beta) = \frac{1}{n} \sum_{i=1}^n \beta_i \cdot \ell(f_{\mathbf{w}}(x_i), y_i)$$

with $\mathcal{L}(\mathbf{w}, \mathbf{1}) = \mathcal{L}(\mathbf{w})$ being the population loss. We let $\mathbf{H} = \nabla_{\mathbf{w}}^2 \mathcal{L}(\mathbf{w})$ be the Hessian of the loss with respect to the weights and write $\mathbf{H}_{B_t} = \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_{\mathbf{w}}^2 \ell(f_{\mathbf{w}}(x_i), y_i)$ for the (averaged) Hessian on a mini-batch B_t .

Hessians naturally appear in several applications, three of which will be the focus of this paper and which will inform our metric for quality of approximation: (1) influence functions, (2) approximate unrolling of training loops, and (3) loss landscape analysis and weight editing. We discuss each of these applications in more detail below. We state the formulae here in each section and refer to the appendix for a further explanation.

Influence functions for training data attribution For data attribution, we can compute **influence scores** that allow us to approximate the importance of a data point for a downstream task

$$\mathcal{I}(z_i, \phi) = -\nabla_{\mathbf{w}} \phi(\mathbf{w}^*)^\top \mathbf{H}^{-1} \nabla_{\mathbf{w}} \ell(f_{\mathbf{w}^*}(x_i), y_i). \quad (1)$$

See Appendix F for more details on the derivation and interpretation of this formula.

Unrolling for training dynamics We can unroll training dynamics

$$w_{t+1} = w_t - \frac{\eta_t}{b} \sum_{i \in B_t} \beta_i \nabla_{\mathbf{w}} \ell(f_{w_t}(x_i), y_i)$$

to compute its first-order approximation

$$\mathbf{J}_t = \frac{dw_{t+1}}{dw_t} = \mathbf{I} - \eta_t \cdot \mathbf{H}_{B_t}. \quad (2)$$

which involved Hessian vector products when multiplied with the gradient $\nabla_{\mathbf{w}} \ell(f_{\mathbf{w}^*}(x_i), y_i)$. A variant can also be used to compute data attribution scores, introduced in [Bae et al., 2024]

$$\tau_{\text{SOURCE}}(\phi, z_i) = \nabla_{\mathbf{w}} \phi(w_T)^\top \sum_{\ell=1}^L \left(\prod_{\ell'=\ell+1}^L \bar{S}_{\ell'} \right) \bar{r}_{i,\ell}. \quad (3)$$

We refer to Appendix G for more details.

Loss landscape analysis for weight editing A third family of applications uses $\mathbf{H}$ not through matrix–vector products but through its *eigendecomposition*: the eigenvectors of $\mathbf{H}$ define a basis for parameter space that ranks directions by curvature, and this ranking is used to select which components of a trained model to keep or discard. Merullo et al. [2025] observe that, when curvature is averaged across many data points, high-curvature directions correspond to weight components shared across inputs (generalisation), while low-curvature directions correspond to weight components used by few inputs (memorisation). They exploit this by projecting MLP weight matrices onto the high-curvature subspace, thereby suppressing verbatim recitation. See Appendix D for the full formulation.

4 To be a Hessian: Approximations

Hessians require $\mathcal{O}(p^2)$ memory, making them infeasible to compute in practice. Any tractable approximation must make cutoffs with regard to that.

4.1 Approximating the Fisher: the Kronecker family

In practice, we need to find a tractable form for the approximations that allow us to both reduce memory cost and make operations such as matrix-vector product or inversion cheap.

Block structure. The first assumption we make is that the Hessian is approximately **block-diagonal**. If we decompose the weights w in terms of its modules $w = w_1 \amalg \dots \amalg w_\ell$ with parameter count $p_i = |w_i|$, we get a decomposition of the Hessian $H : \bigoplus_{i=1}^\ell \mathbb{R}^{p_i} \rightarrow \bigoplus_{i=1}^\ell \mathbb{R}^{p_i}$ in terms of the modules $\mathbf{H}_{ij} : \mathbb{R}^{p_i} \rightarrow \mathbb{R}^{p_j}$. The block-diagonal assumption implies that there is no cross-module interaction, i.e. $\mathbf{H}_{ij} = 0$ if $i \neq j$. This reduces the memory cost from $\mathcal{O}(p^2)$ to $\sum_{i=1}^\ell \mathcal{O}(p_i^2)$, and the computational cost of inversion from $\mathcal{O}(p^3)$ to $\sum_{i=1}^\ell \mathcal{O}(p_i^3)$. The block-diagonal structure is crucial as we can invert the whole matrix by independently inverting the individual blocks.

Kronecker structure. Within each block we make another simplifying assumption. If w_i is a matrix of shape $(d_{\text{in}}, d_{\text{out}})$, then the Hessian is a linear operator $\mathbf{H}_{ii} : \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}} \rightarrow \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$. We say that such a linear operator F admits a Kronecker product factorization, if there are $\mathbf{A} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{in}}}$ and $\mathbf{B} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{out}}}$ such that $F(M) = \mathbf{B} M \mathbf{A}^\top$ and write $\mathbf{F} = \mathbf{A} \otimes \mathbf{B}$. Compared to a general matrix which has $\mathcal{O}(p_i^2) = \mathcal{O}(d_{\text{in}}^2 d_{\text{out}}^2)$ memory cost and $\mathcal{O}(p_i^3) = \mathcal{O}(d_{\text{in}}^3 d_{\text{out}}^3)$ inversion cost, a Kronecker-factored matrix has $\mathcal{O}(d_{\text{in}}^2 + d_{\text{out}}^2)$ memory cost and $\mathcal{O}(d_{\text{in}}^3 + d_{\text{out}}^3)$ inversion cost. We can store $\mathbf{A}, \mathbf{B}$ independently and the inverse of $\mathbf{A} \otimes \mathbf{B}$ is given by $\mathbf{A}^{-1} \otimes \mathbf{B}^{-1}$.

All the approximations of the Hessian we consider in this paper take the form

$$\mathbf{H} \approx \mathbf{P} \approx \mathbf{P}_{ii} \approx (\mathbf{A}_i \otimes \mathbf{B}_i)$$

with different choices of Kronecker factors $\mathbf{A}_i, \mathbf{B}_i$. Additionally, we compute their **Eigenvalue correction** which still has the comparable memory and computational cost

$$\hat{\mathbf{F}}_\ell = (\mathbf{U}_A \otimes \mathbf{U}_B) \hat{\mathbf{\Lambda}}_\ell (\mathbf{U}_A \otimes \mathbf{U}_B)^\top, \quad [\hat{\mathbf{\Lambda}}_\ell]_{ii} = \mathbb{E} \left[((\mathbf{U}_A \otimes \mathbf{U}_B)^\top g_\ell)_i^2 \right]. \quad (4)$$

where $\mathbf{U}_M$ is the eigenvectors of a matrix M .

Methods we benchmark. We benchmark: KFAC, Shampoo, MAC, Identity, and its eigenvalue correction. In toy models we also compute Hessians, Gauss-Newton Hessians, Fisher information, and block variants thereof. See Appendix B for more details.

5 To be a Hessian: Metrics

Based on Section 3 we now discuss what possible metrics are. We group these in four different categories and explain their relation to downstream tasks. We write $\mathbf{H}$ for the Hessian and $\hat{\mathbf{H}}$ for the approximation.

Matrix-level. We can consider the difference of the matrices $D = \mathbf{H} - \hat{\mathbf{H}}$ and compute the norm of this difference. A popular choice is the Frobenius norm, e.g. George et al. [2018] prove that EK-FAC has lower Frobenius norm than K-FAC and Morwani et al. [2025] show that Shampoo is the optimal rank-1 Kronecker factorization under the Frobenius norm for the Fisher information matrix. Gao et al. [2021] instead argue for trace-based alternatives, and matrix cosine similarity $\langle \hat{\mathbf{H}}, \mathbf{H} \rangle_F / (\|\hat{\mathbf{H}}\|_F \|\mathbf{H}\|_F)$ is another common option. These are the metrics historically used to argue for a new approximation. They have no direct downstream application; we use them only as ground-truth references at toy scale.

Hessian vector products $\rightarrow$ unrolling. In practice we will be interested in the action of $\mathbf{H}$ and $\hat{\mathbf{H}}$ on training data gradients v or test data gradients w . Here again we face multiple options. For downstream tasks, we usually care about the dot product of gradients, which motivates two metrics: the cosine similarity $\langle \mathbf{H}v, \hat{\mathbf{H}}v \rangle$ or the difference in dot products $\langle v, \mathbf{H}w \rangle - \langle v, \hat{\mathbf{H}}w \rangle$. Relative versions of these where we normalize by the L_2 -norm of $\mathbf{H}v$ are also possible. HVP-level fidelity is the relevant primitive for approximate unrolling (Section 3), where each step applies $\mathbf{H}$ once and approximation errors compound across thousands of steps rather than averaging away.

Inverse Hessian vector products $\rightarrow$ influence functions. As mentioned in Section F, the inverse Hessian vector product $\mathbf{H}^{-1}\mathbf{v}$ is necessary to compute influence scores. In practice the Hessian $\mathbf{H}$ is not invertible and this confronts us with the question of how the inverse is to be computed: We may either (1) dampen the Hessian $\mathbf{H} + \lambda\mathbf{I}$ or (2) compute the pseudo-inverse $\mathbf{H}_\lambda^+$ by cutting off eigenvalues that are smaller than the threshold λ . In either case, we must decide for a choice of λ , where a big λ will cause the Hessian to be close to the $\mathbf{I}$ or $\mathbf{0}$, and a small λ will not be sufficiently strong to make inversion stable. Either way, we can again choose to compare dot products $\langle \mathbf{H}^{-1}\mathbf{v}, \hat{\mathbf{H}}^{-1}\mathbf{v} \rangle$, relative differences $\langle \mathbf{v}, \mathbf{H}^{-1}\mathbf{w} \rangle - \langle \mathbf{v}, \hat{\mathbf{H}}^{-1}\mathbf{w} \rangle$, or round trip errors $\frac{\|\mathbf{H}\hat{\mathbf{H}}^{-1}\mathbf{v} - \mathbf{v}\|}{\|\mathbf{v}\|}$ [Wang et al., 2026a].

Downstream metrics. For data attribution we compute LDS scores and filtering data to improve/decrease task specific performance. For weight-editing we compute the perplexity of the edited model to measure its general performance and task specific performance.

6 Experiments

6.1 Setup

We perform experiments at three different scales:

- **Small toy models ($\sim 3\text{k parameters}$)** where the exact Hessian is computable. This allows us to compute a wide range of metrics. Having access to the ground truth, the main purpose is to study the correlation between different metrics and to verify whether the decoupling between downstream performance and other metrics is observed at this scale.
- **Medium scale models ($\sim 500\text{k parameters}$)** where the exact Hessian is not computable but where we can compute a wider range of metrics than at large scale. The main purpose is to compute models that undergo full training, while still maintaining reasonable computational cost, and to verify whether the observations at small scale hold at this scale.
- **Large language models ($\sim 7\text{B parameters}$)** where pretraining validations go beyond the scope of academic compute. We are still able to study two setups that provide a valuable signal: (1) We study the LoRA finetuning regime and (2) study the loss landscape through weight editing by sampling from the pretraining set. Here we also take a closer look at the Pareto frontier between accuracy and speed.

6.2 Toy model

We train a small 8-layer MLP on the digits task for 1000 epochs, details can be found in the appendix. After approximately 100 epochs the model converged. First, we observe that the dampened Hessians have low correlations across different damping factors Figure 1(a). This has two consequences: 1. The choice of damping hyperparameter may be more important than the choice of an approximator 2. It is not clear which dampened Hessian should be compared to an approximator, since approximators are typically dampened per layer.

Furthermore, we compared a wide variety of metrics and find that while some metrics exhibit good correlation, other metrics are decoupled. For example, Hessian vector products (HVPs) and inverse Hessian vector products (iHVPs) have low correlation, Figure 1(b). This also is reflected in correlations between different approximators. In particular, all approximators including the Gauss-Newton Hessian exhibit low correlation. Surprisingly, LDS scores Figure 1(c) show that, for this hyperparameter regime, the Hessian performs **worse** and uncorrelated methods perform better. A sweep shows that Hessian LDS scores become competitive, but only after switching the inversion strategy and choosing the correct damping factor.

Finally, we investigate the dependence on training. We approximate the training Jacobian (Section G.0.1) and compute the difference $\|\mathbf{J}_{t:0}(w_0) - \hat{\mathbf{J}}_{t:0}(w_0)\|_2$ where $\hat{\mathbf{J}}_{t:0}$ uses the Hessian approximators in Equation (2). Our results in Section 6.2 indicate that none of the approximations is performing comparatively well and we get no meaningful signal which one may be better.

We finish by noting that our results here are sensitive to different choices: Architecture, inversion strategy, or epoch even if the model has been completely overtrained.

Method	Exact	KFAC	EKFAC	GNH	FIM	Bl. FIM	Bl. Hess.	Shampoo	E-Sh.	MAC	EMAC	Identity
$\ (J - \tilde{J})w_0\ $	0.00	4.04	4.06	4.34	4.48	4.10	4.19	4.07	4.08	5.32	4.10	12.01
$\frac{\ (J - \tilde{J})w_0\ }{\ Jw_0\ }$	0.00	0.32	0.32	0.35	0.36	0.33	0.33	0.32	0.33	0.42	0.33	0.96

Table 1: Approximation error of the training-Jacobian-vector product $\tilde{J} \cdot w_0$ against the exact $J \cdot w_0$ on the toy MLP. All Kronecker methods cluster at rel. $L^2 \approx 0.32$; none of the approximators recovers the exact unrolled gradient.

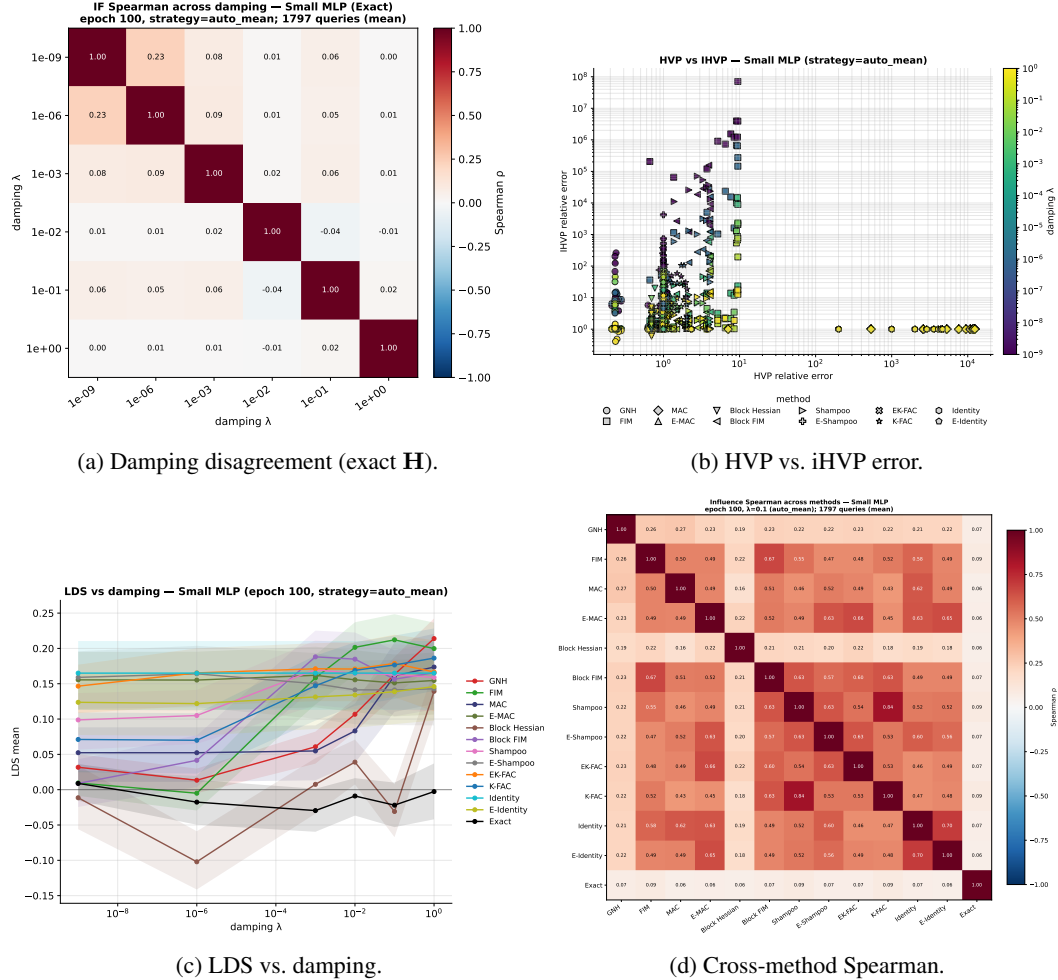


Figure 1: **Toy MLP results.** (a) Spearman correlation of exact-Hessian influence rankings across damping values; the exact Hessian disagrees with itself across λ . (b) HVP-error vs. iHVP-error for each method, showing decoupling between forward and inverse fidelity. (c) LDS as a function of damping for all methods; method choice is dominated by λ across most of the range. (d) Cross-method Spearman heatmap at fixed damping; FIM-family methods cluster, with the exact Hessian as outlier.

208 We expand on each of these dependencies in Appendix C: architecture sensitivity (where MAC pulls
209 ahead on a ResNet variant), the gap between GNH and the exact Hessian, the pseudo-inverse versus
210 damping comparison, and post-convergence drift in influence rankings.

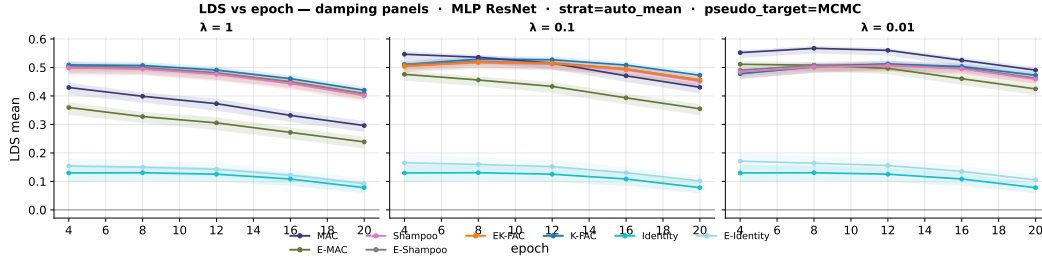


Figure 2: **LDS scores for different approximators.** LDS as a function of training epoch for each method, with damping λ swept in color. MAC produces consistently competitive (and at moderate damping, slightly higher) LDS scores than (E)K-FAC, (E)-Shampoo across the trajectory.

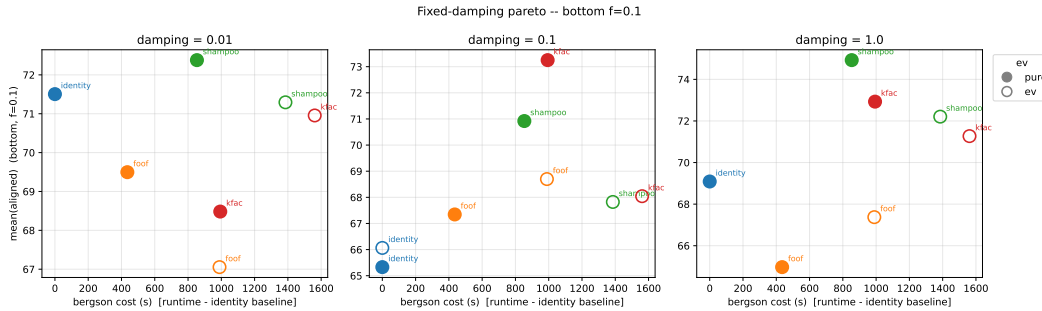


Figure 3: **Emergent misalignment Pareto frontier.** We filter out 0.1 fraction of data to see if we can identify highly misaligned data points. Thus lower mean aligned scores are **better**. Identity and foof (equivalent to MAC here) perform better despite being also cheaper. See Appendix H for the full sweep.

211 6.3 MLP ResNet

212 We move from the small MLP to a larger MLP-ResNet ($\sim 500k$ parameters) where the exact Hessian
 213 is no longer materializable. Across the full damping sweep (Figure 2), MAC produces competitive
 214 LDS scores: At moderate damping it slightly outperforms KFAC, EKFAC and Shampoo, and at
 215 extreme damping it converges with the rest of the FIM-family cluster. The gap is small in absolute
 216 terms but stable across epochs. Notably, FOOF is cheaper to compute. We find this to hold also for a
 217 variant where we replace MLPs with SwiGLUs [Shazeer, 2020].

218 6.4 Language models

219 6.4.1 Language model finetuning attribution

220 At the LLM scale we LoRA-finetune Qwen 2.5 7B [Qwen et al., 2025] on an emergent misalignment
 221 dataset [Betley et al., 2026]. We compare the data attribution scores obtained through different
 222 methods by retraining on filtered datasets and measuring misalignment of those models. We find
 223 that the **cheaper** methods, Identity and MAC, perform better at both identifying data points that
 224 are especially harmful or especially harmless. This holds true across different damping values. See
 225 Figure 3

226 6.4.2 Weight editing

227 We evaluate all Hessian approximations on the memorization-suppression task introduced by Merullo
 228 et al. [2025], who use a KFAC eigendecomposition of OLMo-2-7B [Team, 2024] MLP weights
 229 to project out low-curvature directions and thereby suppress verbatim recitation while preserving
 230 general capabilities (Section 3).

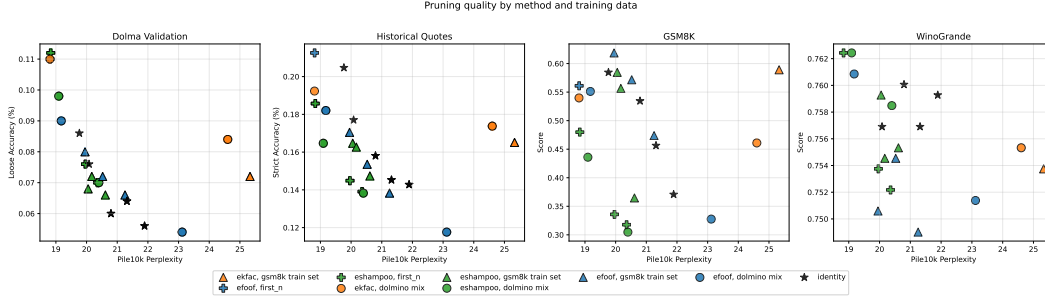


Figure 4: **Weight editing on OLMo-2-7B**. "first_n" refers to the unmixed dolmino training data used in Merullo et al. [2025]. Lower is better on the two left panels (memorization suppression); higher is better on the two right panels (preserved capability). The unedited model has perplexity 17.6, Dolma loose accuracy 1.00, Quotes strict accuracy 0.97, GSM8K 0.64, and WinoGrande 0.77.

Setup. We follow the experimental protocol of Merullo et al. [2025]: we edit layers 23–25 of OLMo-2-7B by retaining eigenvector pairs whose cumulative curvature mass reaches a threshold ρ (see Section 3), and evaluate on four metrics: (1) loose accuracy on the Dolma validation memorization set (lower is better: more memorization suppressed), (2) strict accuracy on a held-out set of historical quotes, (3) GSM8K score (math reasoning), and (4) WinoGrande score (commonsense reasoning). We plot all four against Pile-10k perplexity as a proxy for general coherence.

We depart from Merullo et al. [2025] in two ways. First, we use the OBD saliency ranking described in Appendix D instead of ranking by curvature mass alone. On its own this change has a modest effect when using K-FAC or similar approximations. However, if we use the identity approximation, we see large improvements on GSM8K without significant degradation on other metrics.

Second, we take a closer look at the calibration data used to train the curvature factors. Merullo et al. [2025] stream the first ~ 10 M tokens from Dolmino [AI2, 2024], but because the dataset is organized into subdirectories, this draws exclusively from the first alphabetical subdomain and contains no mathematical data. We test two alternatives: (a) a proportionally weighted mix of all Dolmino subdomains matching the published Dolmino-50B recipe ("dolmino mix"), and (b) training exclusively on the GSM8K train split that is part of Dolmino ("gsm8k train set"), which contains only ~ 2 M tokens.

Results. Figure 4 shows all experiments across three methods (EK-FAC, EFOOF, EShampoo) and all calibration sets, plus the identity baseline. We draw three conclusions:

- Identity is hard to beat.** Despite using no curvature information at all, the identity approximation achieves competitive memorization suppression and near-best GSM8K scores. This is consistent with the observation from Section 3 that curvature fidelity does not straightforwardly predict downstream editing quality.
- EK-FAC preserves capacity better than EFOOF (equivalent to EMAC here) and EShampoo given specific training data.** Among the curvature-based methods, EK-FAC achieves the best trade-off between memorization suppression and perplexity when training using the GSM8K train set, followed by EFOOF.
- Capacity preservation is dataset-dependent; forgetting is not.** Switching from the dolmino mix to the GSM8K train set makes little difference to the memorization suppression metrics (Dolma and Quotes accuracy), but substantially affects the downstream reasoning scores. In particular, EK-FAC trained on the GSM8K train set achieves the best GSM8K score among all curvature-based methods, surpassing even the identity baseline. This suggests that which capabilities survive the edit depends more on the calibration data than on the Hessian approximation, while memorization suppression is robust to this choice.

7 Discussion

Limitations of Frobenius norm as metric. Previous work has shown that gradients live in a low-dimensional subspace of the full parameter space. In that case, we have no reason to apriori expect the top eigenvalue direction to align with the gradient subspace. Thus the Frobenius norm, being agnostic to the distribution of inputs, may not be a good proxy for the performance of the approximation.

Decoupling of HVP and iHVP. As we have seen, there is a large discrepancy between HVP and iHVP. To see why this can happen, consider the perturbation identity

$$\hat{\mathbf{H}}_{\lambda}^{-1} - \mathbf{H}_{\lambda}^{-1} = -\hat{\mathbf{H}}_{\lambda}^{-1} (\hat{\mathbf{H}} - \mathbf{H}) \mathbf{H}_{\lambda}^{-1}. \quad (5)$$

An eigenvalue-direction error of magnitude $\Delta\sigma$ at $\mathbf{H}$ -eigenvalue σ then contributes an iHVP error of order $\Delta\sigma/(\sigma + \lambda)^2$. The amplification factor varies from $1/\sigma^2$ in well-conditioned directions ($\sigma \gg \lambda$) to $1/\lambda^2$ where damping dominates ($\sigma \ll \lambda$); along directions in between, errors are amplified by up to the conditioning number $\kappa(\mathbf{H}_{\lambda})^2$ relative to the HVP.

Are approximators sample-starved? The Kronecker factors $\mathbf{A}_{\ell}, \mathbf{B}_{\ell}$ are estimated from a finite number of forward/backward passes through the frozen model. Importantly, we sample at two stages: (1) We sample input data $x \sim \mathcal{D}$, but we also sample $y \sim f_w(x)$ labels from the model distribution. To our knowledge, a systematic study of sample efficiency is lacking. For a single sweep, we have observed that not sampling simply one label $y \sim f_w(x)$, which is the common technique, but using all labels and weighting them appropriately according to their probability under $f_w(x)$, can improve the quality of the approximation at least on downstream tasks (see Appendix C)

Data attribution metrics have their own blind spots. LDS measures correlation between predicted and actual leave-one-out outcomes on a centered, in-distribution set of training points. In common applications, we are actually interested in the **tail** of the influence scores or clusters of points that are not randomly sampled. For other downstream applications such as data filtering at language level scale having access to ground truth is intractable, therefore making it difficult to compare a method against a gold standard. In this case, we can only compare methods against each other.

Decoupling downstream tasks performance. Benzing [2022] showed that for the purposes of optimization regime that KFAC’s empirical success is well-explained by an activation-only first-order preconditioner, not by curvature approximation. This invites a similar hypothesis in our case: Does the success of (E)K-FAC for the purposes of influence functions root in its approximation quality? Or may there be a different, simpler, reason for the success (e.g. of purely statistical). We have seen empirical hints for such a hypothesis, but cannot come to a conclusion on this point yet.

Limitations. In our experiments, we tried to cover different architectures and different scales. We already observe a large amount of variance and our search is by far not exhaustive. Training on more seeds, refining the downstream tasks by e.g. including more finetuning examples, or even scaling up further are all important points that we did not cover in this work. To meaningfully infer how robust these finding are and if for example simpler methods such as MAC outperform KFAC consistently, we would need to run more ablations.

8 Conclusion

We have investigated the role of Hessian approximations in regimes outside of optimization, where approximation quality may be a more important consideration. Our results show that a variety of important metrics, such as Frobenius norm, unrolling error, and downstream data attribution performance, give a decoupled picture: Metrics are decoupled from each other and we did not find a convincing case where an approximation is convincingly better than others. This stresses the need for a more systematic investigation of the relationship between these approximations, e.g. across sample size, and motivates the search for better approximations e.g. beyond the scope of Kronecker structure. Furthermore, we caution to infer too much from the performance of these approximations on downstream tasks and encourage to look for other reasons why these approximations could perform well.

References

- AI2. Dolmino Mix 1124. <https://huggingface.co/datasets/allenai/dolmino-mix-1124>, 2024.
- Shunichi Amari. Natural gradient works efficiently in learning. *Neural Computation*, 10(2):251–276, 1998.
- Juhan Bae, Nathan Ng, Alston Lo, Marzyeh Ghassemi, and Roger Grosse. If influence functions are the answer, then what is the question? In *Advances in Neural Information Processing Systems*, 2022.
- Juhan Bae, Wu Lin, Jonathan Lorraine, and Roger Grosse. Training data attribution via approximate unrolling. In *Advances in Neural Information Processing Systems*, 2024.
- Nora Belrose and Adam Scherlis. Understanding gradient descent through the training jacobian, 2024. URL <https://arxiv.org/abs/2412.07003>.
- Frederik Benzing. Gradient descent on neurons and its link to approximate second-order optimization, 2022. URL <https://arxiv.org/abs/2201.12250>.
- Jan Betley, Niels Warncke, Anna Sztzyber-Betley, Daniel Tan, Xuchan Bao, Martín Soto, Megha Srivastava, Nathan Labenz, and Owain Evans. Training large language models on narrow tasks can lead to broad misalignment. *Nature*, 649(8097):584–589, January 2026. ISSN 1476-4687. doi: 10.1038/s41586-025-09937-5. URL <http://dx.doi.org/10.1038/s41586-025-09937-5>.
- R Dennis Cook. Detection of influential observation in linear regression. *Technometrics*, 19(1):15–18, 1977.
- Kai-Xin Gao, Xiao-Lei Liu, Zheng-Hai Huang, Min Wang, Zidong Wang, Dachuan Xu, and Fan Yu. A trace-restricted Kronecker-factored approximation to natural gradient. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 7519–7527, 2021.
- Thomas George, César Laurent, Xavier Bouthillier, Nicolas Ballas, and Pascal Vincent. Fast approximate natural gradient descent in a kronecker-factored eigenbasis. In *Neural Information Processing Systems*, 2018.
- Roger Grosse, Juhan Bae, Cem Anil, Nelson Elhage, Alex Tamkin, Amirhossein Tajdini, Benoit Steiner, Dustin Li, Esin Durmus, Ethan Perez, et al. Studying large language model generalization with influence functions. *arXiv preprint arXiv:2308.03296*, 2023.
- Vineet Gupta, Tomer Koren, and Yoram Singer. Shampoo: Preconditioned stochastic tensor optimization. In *International Conference on Machine Learning*, 2018.
- Pang Wei Koh and Percy Liang. Understanding black-box predictions via influence functions. In *International Conference on Machine Learning*, 2017.
- Frederik Kunstner, Lukas Balles, and Philipp Hennig. Limitations of the empirical Fisher approximation for natural gradient descent. In *Advances in Neural Information Processing Systems*, 2019.
- Yann LeCun, John S Denker, and Sara A Solla. Optimal brain damage. In *Advances in Neural Information Processing Systems*, volume 2, 1989.
- James Martens. New insights and perspectives on the natural gradient method. *Journal of Machine Learning Research*, 21(146):1–76, 2020.
- James Martens and Roger Grosse. Optimizing neural networks with kronecker-factored approximate curvature. In *International Conference on Machine Learning (ICML)*, 2015.
- Jack Merullo, Srihita Vatsavaya, Lucius Bushnaq, and Owen Lewis. From memorization to reasoning in the spectrum of loss curvature. *arXiv preprint arXiv:2510.24256*, 2025.
- Depen Morwani, Itai Shapira, Nikhil Vyas, Eran Malach, Sham M. Kakade, and Lucas Janson. A new perspective on shampoo’s preconditioner. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025. URL <https://arxiv.org/abs/2412.15115>.
- Laura Ruis, Maximilian Mozes, Juhan Bae, Siddhartha Rao Kamalakara, Dwaraknath Gnaneshwar, Acyr Locatelli, Robert Kirk, Tim Rocktäschel, Edward Grefenstette, and Max Bartolo. Procedural knowledge in pretraining drives reasoning in large language models. In *The Thirteenth International Conference on Learning Representations*, 2025.

- 363 Nicol N. Schraudolph. Fast curvature matrix-vector products for second-order gradient descent. *Neural*
364 *Computation*, 14(7):1723–1738, 2002.
- 365 Hyunseok Seung, Jaewoo Lee, and Hyunsuk Ko. Mac: An efficient gradient preconditioning using mean
366 activation approximated curvature. *2025 IEEE International Conference on Data Mining (ICDM)*, pages
367 693–702, 2025.
- 368 Noam Shazeer. Glu variants improve transformer, 2020. URL <https://arxiv.org/abs/2002.05202>.
- 369 OLMo Team. 2 OLMo 2 Furious. *arXiv preprint arXiv:2501.00656*, 2024.
- 370 Edward Turner, Anna Soligo, Mia Taylor, Senthoooran Rajamanoharan, and Neel Nanda. Model organisms for
371 emergent misalignment, 2025. URL <https://arxiv.org/abs/2506.11613>.
- 372 Andrew Wang, Elisa Nguyen, Runshi Yang, Juhan Bae, Sheila A. McIlraith, and Roger Baker Grosse. Better
373 training data attribution via better inverse hessian-vector products. In *The Thirty-ninth Annual Conference on*
374 *Neural Information Processing Systems*, 2026a.
- 375 Weiyi Wang, Junwei Deng, Yuzheng Hu, Shiyuan Zhang, Xirui Jiang, Runtong Zhang, Han Zhao, and Jiaqi W.
376 Ma. Taming hyperparameter sensitivity in data attribution: Practical selection without costly retraining. In
377 *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026b.
- 378 Hongbo Zhu and Angelo Cangelosi. Revisiting data attribution for influence functions. *arXiv preprint*
379 *arXiv:2508.07297*, 2025.

380 A Reproducibility, compute, and datasets

381 **Code release.** An anonymized version of our code that reproduces the toy-scale and MLP-ResNet
382 experiments, including the full hyperparameter-sweep configurations, is included as supplemen-
383 tary material: <https://anonymous.4open.science/r/Hessian-Approximation-Toy-Slim-2385>. A non-
384 anonymized release is planned for the camera-ready. The LLM-scale experiments build on the
385 publicly released codebases of Merullo et al. [2025] (weight editing) and the bergson library (factor
386 estimation); we will release our extensions to those codebases at the same time.

387 **Toy MLP and MLP-ResNet hyperparameters.** The toy MLP is an 8-layer fully-connected net-
388 work ($\sim 3k$ parameters) trained on the sklearn `digits` dataset for 1000 epochs with SGD; checkpoints
389 at epochs 10, 100, 500 and 1000 are used in the sweep. The MLP-ResNet ($\sim 500k$ parameters) is
390 trained for 20 epochs on the same task. Across both, we sweep the damping coefficient λ across
391 six orders of magnitude (10^{-9} to 10^0) and report rankings under three inversion strategies (uniform
392 damping, pseudoinverse, and the auto-mean eigenvalue strategy). Full configurations are in the
393 released code.

394 **LLM-scale weight editing protocol.** For weight editing on OLMo-2-7B we follow the protocol
395 of Merullo et al. [2025]: layers 23–25 are edited by retaining eigenvector pairs whose cumulative
396 curvature mass reaches a chosen threshold ρ , evaluated on the four metrics described in Section 6.4.2.
397 For factor calibration we use either (i) the first $\sim 10M$ tokens of the published Dolmino subdo-
398 main ordering (matching Merullo et al. [2025]), (ii) a proportionally weighted mix of all Dolmino
399 subdomains, or (iii) the GSM8K train split alone ($\sim 2M$ tokens).

400 **LLM-Scale finetuning** We build on Turner et al. [2025] and use their setup on Qwen 2.5 7B-
401 Instruct.

402 **LDS protocol.** The Linear Datamodelling Score reported in our toy and MLP-ResNet experiments
403 is computed against 50 retrained models with leave-one-out perturbations of the training set; we
404 report the mean score across query points and shade ± 1 standard error of the mean in figures.

405 **Compute.** Toy and MLP-ResNet experiments run on a single A40 GPU; the entire toy sweep
406 (methods $\times$ damping $\times$ epochs $\times$ inversion strategies) completes in approximately 3 GPU-hours in
407 both cases (The toy model case is more expensive because we compute the Hessian metrics). The
408 LLM-scale weight-editing experiments use $8 \times A40$ nodes and consume approximately 2 node-hours
409 total across calibration, edit, and evaluation passes. For the language model experiments, we ran

experiments on an 8xA100 node. Computing attribution itself via influence functions on the 6k dataset takes approximately 20 node-minutes, the full approximate unrolling pipeline roughly 2.5 node-hours (since we use 6 checkpoints) and evaluation of the results takes approximately 30 minutes using Meta-Llama-3.1-70B-Instruct-AWQ-INT4 as judge run locally on the same node.

Datasets and licenses. sklearn digits (BSD-3-Clause) for toy and MLP-ResNet experiments; Dolma [AI2, 2024] and Dolmino-mix-1124 for OLMo factor calibration (ODC-BY); the Pile-10k subset for perplexity evaluation; GSM8K (MIT) and WinoGrande (CC-BY) for downstream capability evaluation. Memorization evaluation prompts follow the protocol of Merullo et al. [2025]. Fashion-MNIST.

Safeguards. We discuss emergent misalignment and build on previous work Turner et al. [2025] for a responsible sharing of datasets.

B Methods we benchmark

Block Fisher. For a linear layer $\mathbf{W}_\ell \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ with $\mathbf{b}_\ell = \mathbf{W}_\ell \mathbf{a}_\ell$, the per-example gradient is the outer product $\mathbf{G}_\ell = \mathbf{s}_\ell \mathbf{a}_\ell^\top$, where $\mathbf{a}_\ell$ are the activations and $\mathbf{s}_\ell = \partial L[y'] / \partial \mathbf{b}_\ell$ the pseudo-gradients with respect to a label y' sampled from the model distribution $f_w(x)$.¹ The per-layer Fisher block is then

$$\mathbf{P}_{\ell\ell} = \mathbb{E}[\mathbf{G}_\ell \mathbf{G}_\ell^\top] = \mathbb{E}[(\mathbf{a}_\ell \mathbf{a}_\ell^\top) \otimes (\mathbf{s}_\ell \mathbf{s}_\ell^\top)] \in \mathbb{R}^{d_{\text{in}} d_{\text{out}} \times d_{\text{in}} d_{\text{out}}}. \quad (6)$$

The methods below differ in how they approximate $\mathbf{P}_{\ell\ell}$ by a Kronecker product $\mathbf{A}_\ell \otimes \mathbf{B}_\ell$.

K-FAC [Martens and Grosse, 2015]. Assume activations and pseudo-gradients are independent within each layer. Then $\mathbb{E}[(\mathbf{a}\mathbf{a}^\top) \otimes (\mathbf{s}\mathbf{s}^\top)]$ factors as $\mathbb{E}[\mathbf{a}\mathbf{a}^\top] \otimes \mathbb{E}[\mathbf{s}\mathbf{s}^\top]$, giving

$$\mathbf{A}_\ell^{\text{K-FAC}} := \mathbb{E}[\mathbf{a}_\ell \mathbf{a}_\ell^\top], \quad \mathbf{B}_\ell^{\text{K-FAC}} := \mathbb{E}[\mathbf{s}_\ell \mathbf{s}_\ell^\top], \quad (7)$$

the activation and pseudo-gradient covariances.

Shampoo [Gupta et al., 2018, Morwani et al., 2025] and TK-FAC [Gao et al., 2021]. Drop the independence assumption and find factors $\mathbf{A}, \mathbf{B}$ that minimize the Frobenius norm $\|\mathbf{P}_{\ell\ell} - \mathbf{A} \otimes \mathbf{B}\|_F$. Morwani et al. [2025, Corollary 2] show that the optimal rank-1 solution is given by trace-weighted variants of the K-FAC factors,

$$\mathbf{A}_\ell^{\text{Sh}} \propto \mathbb{E}[\|\mathbf{s}_\ell\|^2 \mathbf{a}_\ell \mathbf{a}_\ell^\top]^{1/2}, \quad \mathbf{B}_\ell^{\text{Sh}} \propto \mathbb{E}[\|\mathbf{a}_\ell\|^2 \mathbf{s}_\ell \mathbf{s}_\ell^\top]^{1/2}. \quad (8)$$

Shampoo was originally introduced as an Adagrad approximation; TK-FAC was derived independently from a trace-restriction constraint, and using $\text{tr}(\mathbf{a}\mathbf{a}^\top) = \|\mathbf{a}\|_2^2$ one recovers a single power-iteration step of the SVD that solves the rank-1 problem.

FOOF (MAC) and EMAC [Benzing, 2022]. Decomposing K-FAC further, isolate the activation factor only: *MAC* sets $\mathbf{B}_\ell = \mathbf{I}$, and *EMAC* additionally applies the eigenvalue correction in the activation eigenbasis,

$$\hat{\mathbf{F}}_\ell^{\text{MAC}} = \mathbf{A}_\ell \otimes \mathbf{I}, \quad \hat{\mathbf{F}}_\ell^{\text{EMAC}} = (\mathbf{U}_A \otimes \mathbf{I}) \hat{\mathbf{A}}_\ell (\mathbf{U}_A \otimes \mathbf{I})^\top. \quad (9)$$

These retain only the activation covariance and discard the gradient covariance entirely. Benzing [2022] showed that, in the optimization regime, K-FAC’s empirical success is well-explained by an activation-only first-order preconditioner (FOOF), not by curvature approximation; MAC and EMAC are the natural counterparts of this observation in the accuracy-sensitive regime.

Identity baseline. Finally we include the identity, which also admits a Kronecker factorization as $\mathbf{A}_\ell = \mathbf{I}$ and $\mathbf{B}_\ell = \mathbf{I}$, reducing the iHVP to a gradient inner product.

Eigenvalue correction [George et al., 2018]. Given a Kronecker factorization, eigendecompose its factors as $\mathbf{A}_\ell = \mathbf{U}_A \mathbf{\Lambda}_A \mathbf{U}_A^\top$ and $\mathbf{B}_\ell = \mathbf{U}_B \mathbf{\Lambda}_B \mathbf{U}_B^\top$. keeps the Kronecker eigenvectors $\mathbf{U}_A \otimes \mathbf{U}_B$ but replaces the eigenvalue product $\mathbf{\Lambda}_A \otimes \mathbf{\Lambda}_B$ by the empirical second moments of gradients in this basis. We can compute this for any Kronecker factored matrix.

¹For models trained with cross-entropy loss this means sampling from the softmax of the logits.

450 C Additional toy-scale results

451 We expand on more results that we summarized in the experiments.

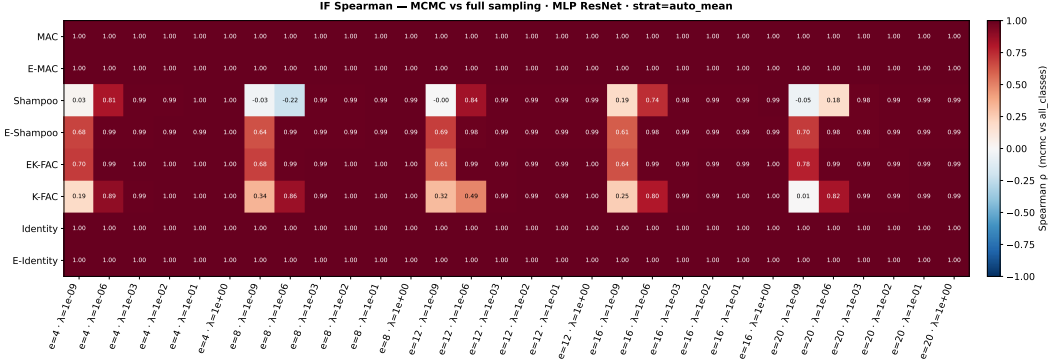
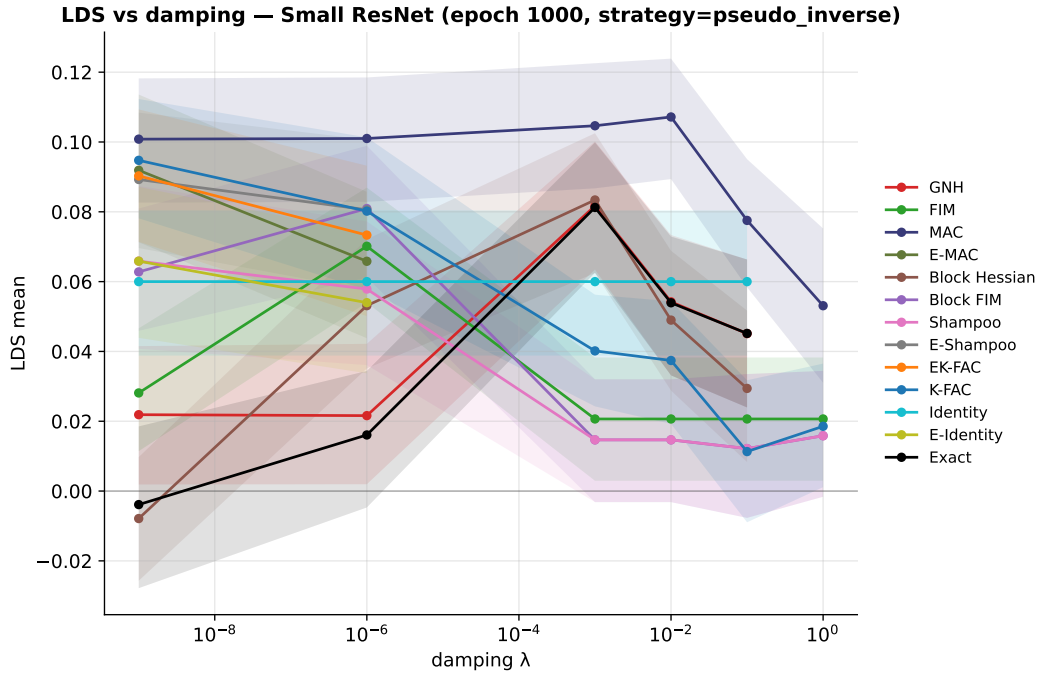


Figure 5: **Sample size sweep (MLP-ResNet)**. Spearman correlation between method rankings as a function of the number of samples used to estimate the Kronecker factors. K-FAC seems to improve if we encourage sample diversity

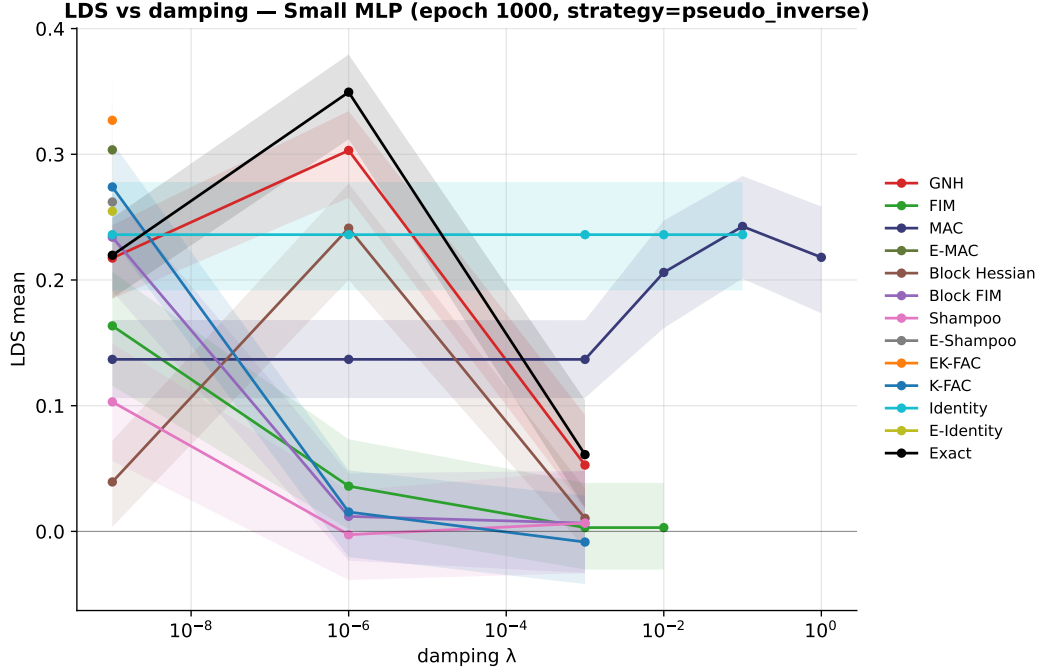


452 D Weight editing: detailed formulation

453 Given a weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and a Kronecker-factored Hessian block $\hat{\mathbf{H}} = \mathbf{B} \otimes \mathbf{A}$
 454 (Section 4.1), we eigendecompose each factor as $\mathbf{A} = \mathbf{U}_A \text{diag}(\boldsymbol{\mu}) \mathbf{U}_A^\top$ and $\mathbf{B} = \mathbf{U}_B \text{diag}(\boldsymbol{\lambda}) \mathbf{U}_B^\top$.
 455 The weight matrix is expressed in the resulting eigenbasis as

$$C = \mathbf{U}_B^\top \mathbf{W} \mathbf{U}_A, \quad C_{ij} = \mathbf{u}_i^\top \mathbf{W} \mathbf{v}_j, \quad (10)$$

456 where $\mathbf{u}_i, \mathbf{v}_j$ are columns of $\mathbf{U}_B, \mathbf{U}_A$ respectively. Each pair (i, j) carries a curvature eigenvalue
 457 $\Lambda_{ij} = \lambda_i \mu_j$.



458 **Curvature-mass selection [Merullo et al., 2025].** Given a retention threshold $\rho \in (0, 1]$, select the
 459 set S of pairs with highest Λ_{ij} until $\sum_{(i,j) \in S} \Lambda_{ij} \geq \rho \sum_{i,j} \Lambda_{ij}$, and reconstruct

$$\mathbf{W}_{\text{edit}} = \mathbf{U}_B (C \odot M) \mathbf{U}_A^\top = \sum_{(i,j) \in S} C_{ij} \mathbf{u}_i \mathbf{v}_j^\top, \quad (11)$$

460 where $M_{ij} = \mathbf{1}[(i, j) \in S]$. This zeros out low-curvature coefficients while preserving the curvature
 461 eigenbasis.

462 **OBD saliency selection.** The criterion above ranks pairs by Λ_{ij} alone, regardless of the magnitude
 463 of C_{ij} . A direction can be retained even if the model places zero weight on it, or discarded even
 464 if the model relies on it heavily. *Optimal Brain Damage* [LeCun et al., 1989] provides a classical
 465 alternative: the second-order Taylor approximation to the loss increase from zeroing a parameter
 466 w_k is $\delta \mathcal{L} \approx \frac{1}{2} h_{kk} w_k^2$. Since the Kronecker eigenbasis diagonalises the Hessian approximation,
 467 $h_{kk} = \Lambda_{ij}$ and the “parameters” are the coefficients C_{ij} . The OBD saliency of pair (i, j) is therefore

$$s_{ij} = \Lambda_{ij} C_{ij}^2 = \lambda_i \mu_j C_{ij}^2, \quad (12)$$

468 and the selection rule becomes: include pairs in descending order of s_{ij} until the desired fraction of
 469 total saliency is reached.

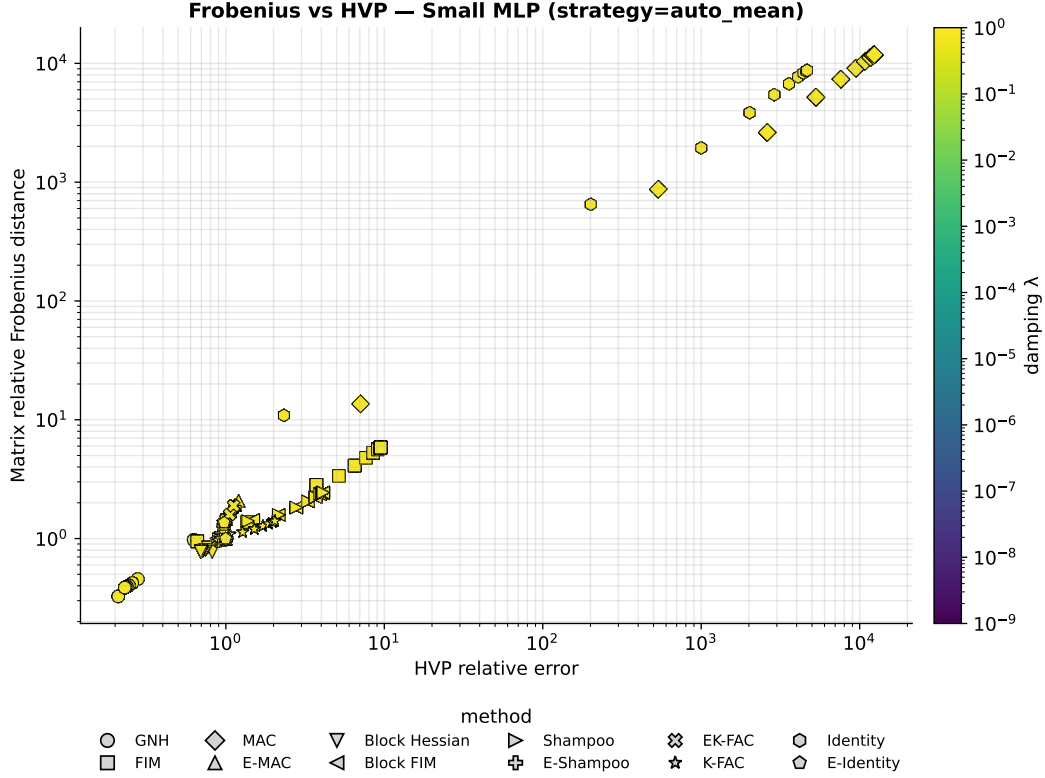
470 E From Hessian to Fisher to Kronecker factors

471 **From H to the Gauss-Newton Hessian G.** For models trained with cross-entropy loss,² the
 472 Hessian admits the decomposition

$$\mathbf{H} = \mathbf{G} + \mathbf{R}, \quad \mathbf{G} = \sum_i [J_w f_w(x_i)]^\top \nabla_f^2 [-\log p(y_i | f)] [J_w f_w(x_i)], \quad (13)$$

473 where $\mathbf{G}$ is the *generalized Gauss-Newton matrix* (GNH) [Schraudolph, 2002] and $\mathbf{R}$ collects
 474 the residual terms involving $\nabla_w^2 f$, i.e. the part of the Hessian that comes from the second-order
 475 dependence of the network outputs f on the parameters. Influence-function methods [Koh and Liang,
 476 2017, Bae et al., 2022, Grosse et al., 2023] use $\mathbf{G}$ in place of $\mathbf{H}$ for two reasons:

²Or generally, when we are in the exponential family regime [Kunstner et al., 2019, Martens, 2020].



1. $\mathbf{G} \succeq 0$ for output-convex losses even when $\mathbf{H}$ is indefinite, side-stepping ill-conditioning from negative curvature off the manifold of minimizers;
2. $\mathbf{R}$ vanishes when f fits y perfectly, so $\mathbf{G}$ is a faithful local surrogate for $\mathbf{H}$ near a well-trained minimum [Martens, 2020, Bae et al., 2022].

From GNH to Fisher. For exponential families, $\nabla_f^2[-\log p(y | f)]$ does not depend on y and equals the Fisher information of the output distribution $p(\cdot | f)$. Substituting into (13) gives

$$\mathbf{G} = \sum_i \mathbb{E}_{\tilde{y} \sim p_w(\cdot | x_i)} \left[\nabla_w \log p_w(\tilde{y} | x_i) \nabla_w \log p_w(\tilde{y} | x_i)^\top \right] =: \mathbf{F}. \quad (14)$$

Two observations matter here. First, the equality $\mathbf{G} = \mathbf{F}$ holds *only after* taking the expectation over $\tilde{y}$ sampled from the model’s predictive distribution at x_i [Martens, 2020, Bae et al., 2022]: the per-example GNH is a deterministic object built from a single forward pass and Jacobian, while the corresponding Fisher summand is the variance of per-example gradients under labels sampled from the model’s own predictions. Second, this reformulation replaces expensive second-order derivatives with cheap first-order ones: a single backward pass on a sampled label $\tilde{y} \sim p_w(\cdot | x_i)$ yields an unbiased estimator of $\mathbf{F}$.

F Influence functions

Influence functions, originally introduced in robust statistics [Cook, 1977], are a method to understand the impact of changes in the training data on a final model w^* . More formally, assume that we have a unique global minimum $w^*(\beta) := \arg \min_w \mathcal{L}(w; \beta)$ and that the trained neural network is $w^* = w^*(1)$.³ The implicit function theorem then allows us to linearly approximate $w^*(\cdot)$

$$\left. \frac{\partial w^*(\beta)}{\partial \beta_i} \right|_{\beta=1} = -\mathbf{H}^{-1} \nabla_w \ell(f_{w^*}(x_i), y_i). \quad (15)$$

³A weaker assumption is sufficient as dTDA shows, but for simplicity we state this simpler version.

We can thus approximate how a perturbation in the dataset, such as removing a data point (x_i, y_i) by setting $\beta_i = 0$, perturbs the weights $\mathbf{w}^*$.

Despite the fact that the formal assumptions do not hold for modern neural networks, Equation (15) can be computed in practice. Koh and Liang [2017] apply this formula first for modern neural networks and Grosse et al. [2023] then scale up the method to language models to study generalization behavior.

In practice, for any differentiable observable $\phi : W \rightarrow \mathbb{R}$ (e.g. the loss on a held-out query z_q), the chain rule gives the *influence on ϕ* :

$$\mathcal{I}(z_i, \phi) = -\nabla_{\mathbf{w}} \phi(\mathbf{w}^*)^\top \mathbf{H}^{-1} \nabla_{\mathbf{w}} \ell(f_{\mathbf{w}^*}(x_i), y_i). \quad (16)$$

We now focus our discussion on the role of the Hessian in this formula. A key obstacle is the computation of the inverse-Hessian-vector product (iHVP) $\mathbf{H}^{-1}v$.⁴ It is established that Hessians are very singular, i.e. most of their eigenvalues are 0, making a strict inversion not possible.

Two adjustments are standard to make the Hessian invertible. First, the Hessian $\mathbf{H}$ is replaced by the Gauss-Newton Hessian. Secondly, $\mathbf{G}$ is then dampened $\mathbf{G}_\lambda = \mathbf{G} + \lambda \mathbf{I}$. Since $\mathbf{G}$ is positive semi-definite, $\mathbf{G}_\lambda$ will be positive definite and thus invertible. To summarize, the new influence formula reads $\mathcal{I}_{GNH}(z_i, \phi) \approx -\nabla_{\mathbf{w}} \phi(\mathbf{w}^*)^\top (\mathbf{G} + \lambda \mathbf{I})^{-1} \nabla_{\mathbf{w}} \ell(f_{\mathbf{w}^*}(x_i), y_i)$.

Bae et al. [2022] give an interpretation of these changes: the dampened Gauss-Newton Hessian is the Hessian of a modified loss function, the proximal Bregman response function

$$\mathcal{L}_{\text{PBRF}}(\mathbf{w}; \beta) = \sum_i D_{\ell_y}(f_{\mathbf{w}}(x_i), f_{\mathbf{w}^*}(x_i)) + \sum_i (\beta_i - 1) \ell(f_{\mathbf{w}}(x_i), y_i) + \frac{\lambda}{2} \|\mathbf{w} - \mathbf{w}^*\|^2, \quad (17)$$

where D_{ℓ_y} is the Bregman divergence of the output-space loss around $f_{\mathbf{w}^*}(x_i)$. Bae et al. [2022] validate their results by successfully predicting the performance of models retrained on $\mathcal{L}_{\text{PBRF}}$. Thus modern influence functions approximate the impact not on the loss landscape of $\mathcal{L}$, but $\mathcal{L}_{\text{PBRF}}$.

We study the impact of these changes. Another alternative to damping is to use the Moore–Penrose pseudoinverse $\mathbf{G}^+$, which inverts only the non-zero eigenvalues. This removes the need to choose λ , but in practice it introduces a new hyperparameter ϵ , used to determine when an eigenvalue is sufficiently close to zero to be ignored. Wang et al. [2026a] report that more accurate Hessian approximations yield better data attribution.

G Unrolling

Another way to understand the impact of training data on training is through differentiation of the training process itself.

G.0.1 Training Jacobian, weight space perturbation

Take a single SGD step with learning rate η_t and mini-batch B_t

$$w_{t+1} = w_t - \frac{\eta_t}{b} \sum_{i \in B_t} \beta_i \nabla_{\mathbf{w}} \ell(f_{w_t}(x_i), y_i). \quad (18)$$

The first order approximation of that training step is given by the Jacobian

$$\mathbf{J}_t = \frac{dw_{t+1}}{dw_t} = \mathbf{I} - \eta_t \cdot \mathbf{H}_{B_t}. \quad (19)$$

Chaining these together, we can consider the **training Jacobian** [Belrose and Scherlis, 2024], the first-order approximation of training $\mathbf{J}_{t:0} = \mathbf{J}_t \cdot \mathbf{J}_{t-1} \cdots \mathbf{J}_0$ allowing us to approximate approximate the impact of perturbation of the initial weights w_0 on the whole training process T :

$$T(w_0 + \epsilon \cdot w') - T(w_0) = \epsilon \cdot \mathbf{J}_{t:0} \cdot w' + \mathcal{O}(\epsilon^2) \quad (20)$$

⁴In practice, one computes $\mathbf{H}^{-1} \nabla_{\mathbf{w}} \phi(\mathbf{w}^*)$ which doesn't affect the final result, but reduces the computational cost.

529 G.0.2 Data space perturbation

530 Instead of perturbation on the initial weights, we can also consider perturbation on the training data
 531 connecting back to our previous section

$$w_{t+1}(\beta) = w_t(\beta) - \frac{\eta_t}{b} \sum_{i \in B_t} \beta_i \nabla_w \ell(f_{w_t}(x_i), y_i) \quad (21)$$

532 and we can again differentiate by β to get an explicit formula for the impact of a training point (x_i, y_i)
 533 on the training trajectory. Differentiating at $\beta = \mathbf{1}$ gives the recursion

$$\frac{\partial w_{t+1}}{\partial \beta_i} = J_t \frac{\partial w_t}{\partial \beta_i} - \frac{\eta_t}{b} \mathbf{1}[i \in B_t] \nabla_w \ell(f_{w_t}(x_i), y_i),$$

534 with initial condition $\partial w_0 / \partial \beta_i = 0$ and J_t as in Equation (2). Unrolling the recursion gives

$$\left. \frac{\partial w_T(\beta)}{\partial \beta_i} \right|_{\beta=\mathbf{1}} = - \sum_{t=0}^{T-1} \frac{\eta_t}{b} \mathbf{1}[i \in B_t] \mathbf{J}_{(t+1):T} \nabla_w \ell(f_{w_t}(x_i), y_i). \quad (22)$$

535 To amortize costs, [Bae et al., 2024] suggest SOURCE, a stage-wise approximation to avoid comput-
 536 ing Hessians at every step. Dividing training into L segments of length K_ℓ with average learning
 537 rate $\bar{\eta}_\ell$ and average mini-batch Hessian $\bar{\mathbf{H}}_\ell$, the segment Jacobian becomes $\bar{S}_\ell = \exp(-\bar{\eta}_\ell K_\ell \bar{\mathbf{H}}_\ell)$
 538 and the within-segment response is $\bar{r}_{i,\ell} = \frac{1}{n} (I - \bar{S}_\ell) \bar{\mathbf{H}}_\ell^{-1} \bar{g}_{i,\ell}$ for $\bar{g}_{i,\ell}$ the average gradient of z_i over
 539 segment ℓ . The full SOURCE attribution is

$$\tau_{\text{SOURCE}}(\phi, z_i) = \nabla_w \phi(w_T)^\top \sum_{\ell=1}^L \left(\prod_{\ell'=\ell+1}^L \bar{S}_{\ell'} \right) \bar{r}_{i,\ell}. \quad (23)$$

540 In either case, a big difference to Section F is that we compute Hessian vector products and thus
 541 avoid inversion

542 H More LLM results

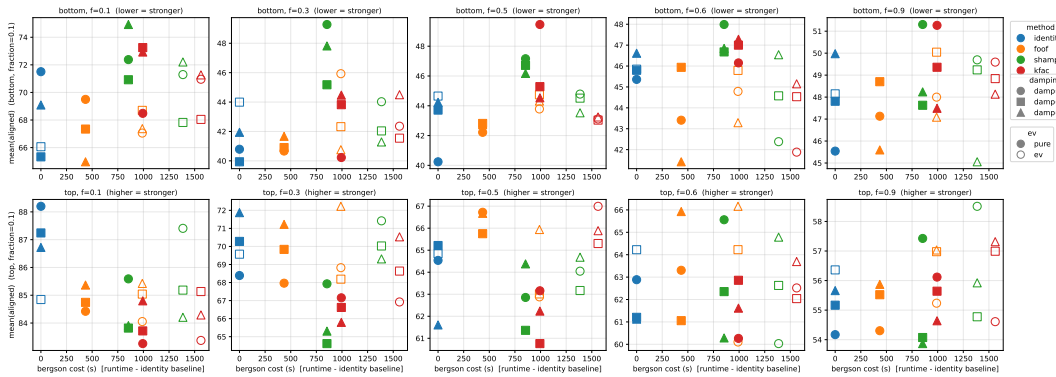


Figure 6: Full hyperparameter sweep across damping and different data filtering proportions.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive” or “we were unable to find the license for the dataset we used”). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”.
- Keep the checklist subsection headings, questions/answers and guidelines below.
- Do not modify the questions and only use the provided macros for your answers.

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: Yes, they are a faithful summary of our findings.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Yes, we discuss limitations in the Discussion section.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: We primarily revise theoretical results and hence refer to the original work for complete proofs. We state the assumptions that are needed.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: We release our code for toy model experiments with a reproduction pipeline and plan to release the language model scale experiments.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: Toy model code base has all the hyper parameters. For language model experiments we build on previous work or plan to release the details in the code base as well.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: We highlight that our results vary across hyper parameters and refer to the code base for the large sweep specifications.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: For computation of LDS scores we report the number of retrainings and describe relevant statistics.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: We indicate so in the appendix.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.

- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

Answer: [Yes]

Justification: We respect copyright of datasets and ensure reproducibility of results.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [No]

Justification: Our results are foundational and don't provide novel methods for downstream applications. Better methods to approximate Hessians may improve performance on downstream tasks, some of which can have negative impact (e.g. poisoning training datasets).

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [Yes]

Justification: All our experiments, except for reproducing emergent misalignment, does not pose a risk. For our emergent misalignment results we refer to previous work which does provide the necessary safe guards.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Where appropriate we cite the used data sets and code bases.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: We release our own research code, but don't release novel libraries or datasets.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: No human subjects were involved.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: No human subjects were involved.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: No LLM was involved in development of core parts of this research.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.